



Project Report

Disaster Management System

1. Introduction

Disasters such as floods, cyclones, wildfires, and landslides cause massive destruction to life and property. Quick access to accurate information and efficient management is crucial in mitigating their effects.

The **Disaster Management System** is a web-based application that provides real-time disaster tracking, visualization, and management features. It consists of a **Django REST Framework backend** for handling data and a **React frontend** for an interactive dashboard.

This project aims to provide a platform where administrators can manage disaster data, and users can view interactive dashboards with insights, maps, and alerts.

2. Objectives

- To build a scalable and interactive **web-based disaster management system**.
- To enable **real-time tracking** and visualization of disasters.
- To provide a **user-friendly dashboard** with charts, maps, and filters.
- To allow administrators to manage disasters through a **customized admin panel**.

- To implement features like **dark mode, notifications, and search filters** for better user experience.
 - To explore **future enhancements** like real-time updates, predictive analytics, and mobile support.
-

3. System Architecture

The system follows a **client-server architecture**:

- **Frontend (React.js + TypeScript)**
 - Displays disaster data interactively.
 - Provides charts, maps, and filters.
 - Includes animations, dark mode, and responsive UI.
 - **Backend (Django + Django REST Framework)**
 - Provides RESTful APIs for disasters.
 - Handles authentication and admin management.
 - Stores disaster records with fields like name, type, location, severity, status, casualties, description, and timestamps.
 - **Database (SQLite / PostgreSQL)**
 - Stores structured disaster data.
-

4. Modules

a) Frontend (React Dashboard)

- **Charts & Graphs:** Disaster distribution by type, severity, and status.
- **Map Integration:** Display disaster locations with Leaflet/Google Maps API.
- **Timeline View:** Show disaster progression over time.
- **Filters & Search:** Filter by type, location, severity, or date.

- **Dark Mode / Light Mode** toggle.
- **Interactive Background** with disaster images slideshow.
- **Animations & Notifications** for enhanced user engagement.

b) Backend (Django + DRF)

- **API Endpoints** for CRUD operations on disasters.
- **Data Fields:** name, disaster_type, location, severity, status, casualties, relief_measures, description, created_at, updated_at.
- **Authentication & Authorization** for secure access.
- **Admin Panel** with filters, search, and export functionality.
- **File Uploads** (e.g., disaster images, reports).

c) Future Enhancements

- **WebSockets/Django Channels** for real-time updates.
 - **Push Notifications** for urgent alerts.
 - **AI/ML Predictions** for risk forecasting.
 - **NLP News Integration** for automatic disaster updates.
 - **Mobile-Friendly PWA** for cross-device accessibility.
-

5. Technologies Used

- **Frontend:** React.js, TypeScript, Recharts, Leaflet, Framer Motion, React Icons.
 - **Backend:** Django, Django REST Framework, Django CORS Headers.
 - **Database:** SQLite (dev), PostgreSQL (production).
 - **Tools:** GitHub, VS Code, Postman.
 - **Deployment Options:** PythonAnywhere / Heroku / AWS.
-

6. Implementation Details

- **Step 1:** Set up Django backend with REST APIs (/api/disasters/).
 - **Step 2:** Create disaster models with advanced fields (severity, status, casualties).
 - **Step 3:** Configure Django Admin for data management.
 - **Step 4:** Build React frontend to fetch and display disasters.
 - **Step 5:** Add charts, maps, filters, and background slideshow.
 - **Step 6:** Implement authentication and role-based access.
 - **Step 7:** Optimize UI with dark mode, animations, and responsiveness.
-

7. Results

- A functional **Disaster Dashboard** with interactive charts and maps.
 - A **Django Admin Panel** for managing disaster records.
 - A user-friendly frontend with **search, filters, dark mode, and background slideshow**.
 - APIs successfully fetching and displaying data across frontend and backend.
-

8. Conclusion

The Disaster Management System successfully demonstrates the integration of **Django REST Framework and React** to create a modern, interactive, and scalable web application. It provides real-time visualization of disasters, efficient data management, and a responsive dashboard for end-users.

Future work includes real-time updates, predictive analytics using AI, and mobile-friendly deployment.

9. References

- Django Documentation (<https://docs.djangoproject.com/>)
- Django REST Framework (<https://www.django-rest-framework.org/>)
- React.js (<https://react.dev/>)
- Recharts & Leaflet Documentation

TEAM MEMBERS

Yuvraj Singh

Mohit Saini

Akshey Kaushik